МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №4

з дисципліни «Методи та технології інженерії даних»

на тему: «Надійне отримання даних із файлів і REST API»

Виконав:
Студент 3 курсу
групи ФЕП-34
Полянський Тимотей

Перевірив:
ас. Фостяк Маркіян Романович

Львів 2026

1. Мета, середовище та вхідні дані

Мета: отримати повний набір даних із файлового пакета CityFlow і REST API, зберегти первинні відповіді (raw), коректно обробити тимчасові помилки та повторно запускати процес без дублікатів у PostgreSQL. Ланцюжок: файловий пакет + API → raw-копії → технічне staging.

Параметр	Значення
Студент	Полянський Тимотей, група ФЕП-34, `student_id: S08`
Середовище	Python 3.14.7 (uv; `requests` 2.32.5, `psycopg` 3.3.6, `pytest` 9.1.1, сервер `fastapi` 0.128.2 + `uvicorn` 0.48.0 — версії з `requirements-lab04.txt`); PostgreSQL 16.15 у Docker (`postgres:16-alpine`, БД `cityflow_s08`)
Репозиторій	github.com/TimoteyPolyanskiy/data-engineering-cityflow-S08, гілка `lab/04-ingestion`, Pull Request: pull/4
Файловий пакет	`cityflow_S08_batch_01_normal_v1.zip`, SHA-256 `84017962ee62af3ba7c05c2108fd518eab5457fce5b4356a0068353d3ae3744c` — збігається з `package_sha256` у `expected_counts.json`. Архів варіанта `CityFlow_Lab04_S08.zip`: `147bfaaf…0598879ab8`
API	Локальний CityFlow API v1.0, `GET /api/v1/delivery-events`, ключ у заголовку `X-API-Key`. Фіксоване вікно з `settings.json`: `occurred_at` від `2026-09-01T00:00:00Z` до `2026-09-07T23:59:59.999999Z`, `available_before = 2026-09-08T23:59:59Z`, `limit = 100`, `incident = false`
Секрети	`credentials.env` скопійовано як `.env.lab04` (у `.gitignore` разом з `data/lab04_variant/`, `data/raw/lab04/`, `credentials.env`, `api_keys.json`); код читає ключ і `DATABASE_URL` лише звідти, у логи, raw-метадані й звіт вони не потрапляють
Нові об’єкти БД	Лише `stg.lab04_records` і `audit.lab04_runs` (`init-db`); `core` (5996 замовлень, 14027 позицій) і `lab03_cf` не змінювались

2. Реалізація трьох функцій (`src/lab04/student.py`)

`read_file_batch()` відкриває ZIP через `zipfile.ZipFile` лише для читання і бере `PACKAGE_INFO.json`: `student_id` має дорівнювати `S08` (варіант визначається за manifest, а не за назвою архіву). Для кожного з 7 файлів manifest перевіряє наявність, довжину байтів (`size_bytes`) і SHA-256; будь-яка розбіжність — `ValueError` з кодом (`MISSING_FILE`, `INVALID_SIZE`, `INVALID_CHECKSUM`). Потім `orders.csv` і `order_items.csv` читаються через `csv.reader` (UTF-8 з BOM, кома): заголовок має точно збігатися з `CSV_COLUMNS` за назвами й порядком, кожен рядок — мати стільки ж полів. Значення лишаються рядками, порожні поля не замінюються нулем, рядки не пропускаються.

`request_page()` робить лише `session.get(..., timeout=(3, 10), allow_redirects=False)` — не більше 4 спроб загалом. HTTP 200 повертається як `Response`. Коди 429, 500, 502, 503, 504, а також `Timeout` і `ConnectionError` повторюються з паузами 1, 2, 4 с; для 429 пауза — більша з backoff і `Retry-After`; очікування понад 30 с завершує запуск помилкою. `SSLError` перехоплюється першим (він є підкласом `ConnectionError`) і не повторюється; 400, 401, 403, 404, 422, перенаправлення та інші коди — одразу виняток. Пауза — через переданий `sleep(delay)`; у лог пишуться номер спроби, код або тип помилки й пауза, без ключа й заголовків.

`fetch_pages()` копіює `params`, починає без `cursor` і після кожного HTTP 200 викликає `save_response()` до розбору JSON. Для кожної сторінки перевіряє: `student_id`, `schema_version` ∈ {1, 2}, `data` — список об’єктів з непорожнім рядковим `event_id`, `count = len(data)`, `total_count` — невід’ємне ціле й незмінне між сторінками, `next_cursor` — `null` або непорожній рядок, що не повторюється. Події додаються без видалення повторів і з усіма полями. Обхід закінчується лише на `next_cursor = null` (максимум 10 000 сторінок), наприкінці кількість отриманих записів має дорівнювати `total_count`.

Транзакційна межа наданого завантажувача. `main.py run` спершу окремою операцією пише `RUNNING` в `audit.lab04_runs`, потім отримує обидва джерела в raw. `prepare_records()` рахує `received`, `unique`, `duplicates_in_input` і зупиняє запуск, якщо той самий ключ має інший payload. `publish()` в **одній транзакції** завантажує записи в тимчасову таблицю, перевіряє конфлікти з уже збереженими payload, виконує `INSERT … ON CONFLICT DO NOTHING` і ставить `SUCCESS`. Збій до `COMMIT` відкочує і вставки, і `SUCCESS`; raw-файли при цьому не входять у транзакцію й лишаються.

3. Файловий пакет

`pytest -q tests/lab04 -k file` — **5 passed**. Команда `file` скопіювала ZIP у `data/raw/lab04/<run_id>/source.zip`, прочитала **420** записів `orders` і **966** записів `order_items` і повернула `EXTRACTED_ONLY` (у БД не пише). Усі 7 файлів manifest пройшли перевірку розміру й SHA-256. Перевірка на копіях реального пакета (`scripts/lab04/check_broken_package.py`): змінений один байт у `refunds.csv` (файл, який ми не завантажуємо) → `INVALID_CHECKSUM:refunds.csv`; пакет без `order_items.csv` → `MISSING_FILE:order_items.csv`; оригінальний пакет із чужим `student_id` → `INVALID_STUDENT_ID`. Неповний або змінений пакет успішного результату не дає.

4. API: повтори, пагінація, raw

Перший запуск `api` після старту сервера (з `run.log`):

Сторінка	Спроба	Відповідь	Пауза	Пояснення
1	1	HTTP 429	1 с	`Retry-After: 1`, backoff після 1-ї невдачі — 1 с; більше з двох = 1 с
1	2	HTTP 200	—	сторінку збережено як `page_0001.json` + `.meta.json`
2	1	HTTP 500	1 с	тимчасова помилка сервера, повтор
2	2	HTTP 200	—	далі сторінки 3–41 — з першої спроби

Отримано **41 сторінку** (40 × 100 + 64), сума `count` = **4064** = `total_count`, різних `event_id` — **4064** (повторів у варіанті S08 немає; окремо в БД `COUNT(DISTINCT payload->>'event_id') = 4064`). Команда `api` повернула `EXTRACTED_ONLY`; значення збігаються з `expected_counts.json` (`api_records` 4064, `api_unique_events` 4064, `api_pages_at_limit_100` 41). У `page_NNNN.meta.json` зберігаються параметри без ключа, `request_id`, час отримання й SHA-256 тіла; байти тіла записуються такими, як їх отримав `requests`, без повторного `json.dumps`.

Тести: `pytest -q tests/lab04 -k "retry or timeout or attempts or tls"` — **11 passed**; повний набір `pytest -v tests/lab04` — **26 passed** (файли тестів не змінювались).

5. Збій → успіх → повтор

До першого успішного запуску в `stg.lab04_records` було **0** записів, в `audit.lab04_runs` — 0 рядків.

Команда	`run_id`	Статус	Нових записів (`first_run_id`)	Staging після
`run --fail-before-commit`	`493a2615-1438-428b-bf0b-83e1b6b4800e`	FAILED	0	0
`run` (1)	`3fdd023e-b4ee-469f-8e20-27a4d6bb88f7`	SUCCESS	5450	5450
`run` (2)	`80fa2c44-ac54-4626-b538-b31e8e355fcc`	SUCCESS	0	5450

Контрольований збій завершився кодом 1 з `error_code = INJECTED_BEFORE_COMMIT`; `sql/lab04/01_verify.sql` показав 0 записів цієї спроби, а її raw-каталог (`source.zip`, 41 сторінка з метаданими, `run.json`, `run.log`) залишився.

Джерело	Received	Unique	Duplicates in input	Inserted (run 1)	Inserted (run 2)	Already present (run 2)
`orders`	420	420	0	420	0	420
`order_items`	966	966	0	966	0	966
`delivery_events`	4064	4064	0	4064	0	4064

Після кожного успішного запуску `verify` дав `ok: true` (orders 420, order_items 966, delivery_events 4064 — як `staging_unique` у `expected_counts.json`). Повтор того самого пакета й API-вікна не додав бізнес-записів; з'явились лише новий рядок журналу й новий raw-каталог, що за методичкою допустимо.

6. Пояснення своїми словами

1. **Чому завершувати обхід після першої короткої сторінки неправильно?** Розмір сторінки не є сигналом кінця: сервер може повернути коротку сторінку посередині, а повна сторінка може бути останньою. Кінець визначає лише `next_cursor = null`; додатково сума записів звіряється з `total_count`.
2. **Чим raw-відповідь відрізняється від JSONB payload у staging?** Raw — точні байти тіла відповіді з SHA-256 і параметрами запиту; їх можна перевірити й обробити повторно. JSONB — розібране значення одного запису без початкових пробілів, порядку ключів і форматування, тому воно не доводить, що саме надіслало джерело.
3. **Чому не можна мовчки пропустити однаковий ID з іншим payload?** Це конфлікт: запис змінився або джерело помилилось. Якщо залишити перший варіант, зміна загубиться непомітно, тому запуск зупиняється (`CONFLICT_IN_INPUT`, `CONFLICT_WITH_TARGET`).
4. **Що означає SUCCESS і чого він не доводить?** Пакет і всі сторінки отримано й технічно перевірено, усі унікальні записи є в staging, і це зафіксовано однією транзакцією. SUCCESS не доводить бізнес-коректності сум, статусів і зв'язків (ЛР №5) та не гарантує, що джерело віддало всі події, яких ми не бачили.

7. Висновки

1. Файловий блок перевіряє весь пакет, а не лише два потрібні CSV: усі 7 файлів manifest звірено за розміром і SHA-256, прочитано 420 замовлень і 966 позицій без зміни значень; змінений або неповний пакет відхиляється.
2. API-блок прочитав 41 сторінку й 4064 події за курсором; навчальні збої 429 і 500 подолано одним повтором із паузою 1 с, а постійні помилки (401, 404, TLS тощо) за тестами не повторюються. Усі 26 відкритих тестів пройдено.
3. Надійність підтверджено на власній БД: збій перед `COMMIT` залишив staging порожнім (0 → 0, FAILED), перший запуск додав 5450 записів, повторний — 0, `verify` — `ok: true`; `core` і `lab03_cf` не змінювались.

8. Використання ШІ

Інструмент: Claude. Поставлені завдання: підготувати чернетки трьох функцій `student.py` (перевірка пакета за manifest, GET-запити з обмеженими повторами, обхід сторінок за курсором) за контрактом методички й відкритими тестами; заповнити `results.md` числами з власних запусків. Внесок автора: постановка завдань, перевірка результатів запусків і пояснення коду. Перевірка: 26 відкритих тестів без змін — passed; реальний сценарій «збій → успіх → повтор» на локальному API й PostgreSQL дав 0 / 5450 / 0 нових записів, кількості збіглися з `expected_counts.json`; окремо перевірено відхилення зміненого й неповного пакета. Виправлена помилка: у першій версії `results.md` для групи тестів повторів було вказано 12 замість фактичних 11; виправлено за збереженим виводом pytest. Цікавий невдалий кейс: під час огляду структури `api_keys.json` Claude вивів у сесію сам ключ, бо він є назвою поля JSON. Ключ діє лише для локального навчального сервера; у Git, звіт і логи він не потрапив, що підтверджено скануванням відстежуваних файлів.

9. Контрольні запитання

1. **Чому HTTP 200 не означає, що отримано всі дані?** 200 підтверджує успіх лише одного запиту — однієї сторінки. Набір може складатися з багатьох сторінок (у нас 41), і отримано його лише тоді, коли пройдено всі сторінки до `next_cursor = null`, а кількість записів дорівнює `total_count`.
2. **Чим count відрізняється від total_count та кількості різних event_id?** `count` — кількість записів на поточній сторінці (100 або 64). `total_count` — кількість усіх записів вибірки разом із повторами (4064). Кількість різних `event_id` рахується після отримання; вона менша за `total_count`, якщо джерело віддало ту саму подію кілька разів (у S08 обидва значення — 4064).
3. **Чому 401 і 429 обробляються по-різному?** 401 — ключ відсутній або неправильний; повтор того самого запиту дасть ту саму помилку, тому зупиняємось одразу. 429 — тимчасове обмеження частоти: після паузи (не меншої за `Retry-After`) запит зазвичай проходить, тому його повторюємо.
4. **Що доводить SHA-256, а чого не доводить?** Збіг SHA-256 доводить, що байти файла ті самі, що й під час створення manifest: файл не змінено й не пошкоджено. Він не доводить, що дані всередині правильні (суми, статуси, зв'язки) і що сам manifest описує повний набір даних джерела.
5. **Чому після відкату raw лишається?** Raw-файли пишуться на диск поза транзакцією БД, і ROLLBACK їх не стосується. Їх лишають навмисно: це доказ того, що саме отримано в цій спробі, за ним можна розібрати причину збою й повторно обробити дані без нового запиту до джерела.
6. **Чому ON CONFLICT DO NOTHING без порівняння payload може приховати проблему?** Він пропускає будь-який запис з уже наявним ключем, навіть якщо його вміст змінився. Тоді нова версія мовчки відкидається, а звіт показує успіх; тому наданий `publish()` спершу порівнює payload і зупиняє запуск при розбіжності.